Hannah D'Cruz
Dr. Li
CS 2123
8 Oct 2025

Counting Inversions

The divide-and-conquer based algorithm to count the number of inversions is significantly faster than the iterative algorithm. This verifies the fact that the algorithmic efficiency of the divide-and-conquer algorithm is O(nlgn), while the algorithmic efficiency for the iterative brute force algorithm is $O(n^2)$. An inversion is when the order of two arrays is not the same, and an element that appears before a second element in one array appears after it in another array. This can be represented in one array by using the indexes of the values in the first array in the order of the values in the second array. Then, anywhere $i < j$ but $a_i > a_j$, the number of inversions increases by one.

The brute force algorithm has an algorithmic efficiency of $O(n^2)$ because it checks every element in the array, which is O(n) time. It compares that element to all the following elements in the array to see if they are smaller, which is a nested O(n) time. If they are, it increments the inversion counter, which is only O(1) time. This is an efficiency of $O(n^2)$.

The divide-and-conquer algorithm has an algorithmic efficiency of O(nlgn), O(lgn) for the divide step and O(n) for the merge step. The algorithm first splits the array in half continuously until each segment is zero or one element long, which is O(lgn). As it reassembles the halves in order, it compares the smallest element from the left and right sides. If the right side is smaller, it is an inversion, so it increments the inversion counter by all the numbers still on the left side. It does this once for every element on every level as it merges, which is O(n). This leads to a runtime of only O(nlgn), while counting the same number of inversions as the brute force algorithm.

	Divide-and-conquer		Brute force	
Number of elements	Inversions	runtime (sec)	Inversions	runtime (sec)
1000	251883	0	251883	0.047127
2000	999757	0.005454	999757	0.101863
4000	4019941	0.003839	4019941	0.474556
8000	15976739	0.031763	15976739	1.947233
16000	64262986	0.046493	64262986	7.64209
32000	256640648	0.109763	256640648	30.34965